A Real Time Project Report (CS456PC)

on

Group Live Chat Application

Submitted

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering - (DS)

By

BATTI ARUN KUMAR (23261A6710)

EEDULA TARAK SAINATH REDDY (23261A6721)

Under the guidance of

DR. BARNALI GUPTA BANIK
(Assoc. Professor)

Ms. D.DEEPIKA
(Asst. Professor)



Department of Emerging Technologies

Mahatma Gandhi Institute of Technology (Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad) Kokapet(V), Gandipet(M), Hyderabad. Telangana - 500 075

2024-25

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University

Hyderabad) GANDIPET, HYDERABAD – 500075, Telangana

CERTIFICATE

This is to certify that the project entitled —" **GROUP LIVE CHAT APPLICATION"** is being submitted by BATTI ARUN KUMAR (23261A6710), EEDULA TARAK SAINATH REDDY (23261A6721), in partial fulfillment of the requirements for the Real Time Project (CS456PC) in **COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)** is a record of Bonafide work carried out by them under our guidance and supervision.

The results of the investigations enclosed in this report have been verified and found satisfactory.

PROJECT GUIDE

Dr. BARNALI GUPTA BANIK
(Assoc. Professor)
Department of E.T

Ms. D.DEEPIKA
(Asst.Professor)
Department of E.T

HEAD OF THE DEPARTMENT

Dr. M. Ramabai
Professor & HOD
Department of E.T

External Examiner

DECLARATION

This is to certify that the work reported in this project titled —" **GROUP LIVE CHAT APPLICATION"** is a record of work done by our team in the Department of Emerging Technologies, Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by our team and not copied from any other source.

Batti Arun kumar
(23261A6710)

Eedula Tarak Sainath Reddy
(23261A6721)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We would like to express our sincere thanks to Prof**. G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. M. Ramabai, Professor and HoD**, Department of Emerging Technologies, MGIT, for all the timely support and valuable suggestions during the period of the project.

We are extremely thankful and indebted to our internal guides **Dr. Barnali Gupta Banik**, **Associate Professor, Ms. D. Deepika, Assistant Professor,** Department of Emerging Technologies, for their constant guidance, encouragement and moral support throughout the project.

We are extremely thankful to **Mr. R. Srinivas, Assistant Professor, Dr. SK. Irfanbabu, Assistant Professor,** Department of Emerging Technologies, MGIT, major project coordinator for their encouragement and support throughout the project.

Finally, we would also like to thank all the faculty and staff of the Emerging Technologies Department who helped us directly or indirectly to complete this project.

Batti Arun kumar

(23261A6710)

Eedula Tarak Sainath Reddy

(23261A6721)

TABLE OF CONTENTS

CERTIFICATE i

DECLARATION ii

ACKNOWLEDGEMENT iii

LIST OF FIGURES vi

LIST OF TABLES vii

ABSTRACT viii

1. INTRODUCTION 1

1.1 Motivation

1.2 Problem Definition

1.3 Existing System

1.4 Proposed System

1.5 Requirements Specification

1.5.1 Software Requirements

1.5.2 Hardware Requirements

1.6 Languages Used

1.6.1 HTML

1.6.2 CSS

1.6.3 Node js

2. LITERATURE SURVEY 5

3. DESIGN METHODOLOGY 8

3.1 Technologies Used

3.2 Flow Chart Diagram

3.3 Working Application Architecture

3.4 User Interface

4. IMPLEMENTATION AND RESULTS 13

5.1 Login/Register Page

5.2 Chat Room Page

5. CONCLUSION AND FUTURE SCOPE 15

BIBLIOGRAPHY 16

APPENDIX 17

LIST OF FIGURES

Figure No	Name of the Figure	Page No
3.2	Flow Chart Diagram for Chat Application	9
5.1	Login/Register Page of Chat Application	13
5.2	Chat Room page of Chat Application	14

LIST OF TABLES

Table No	Name of the Table	Page No
Table 2.1	Literature Survey	7

ABSTRACT

Chat Application: In the modern digital age, real-time communication plays a vital role in connecting individuals and fostering collaboration. The Group Live Chat Application is designed to provide a seamless and interactive messaging platform that enables multiple users to communicate simultaneously in shared chat rooms. This application supports essential features such as real-time message broadcasting, user authentication, typing indicators, and timestamped messages, ensuring a smooth and engaging user experience.

The application utilizes modern web technologies, including **Flask/Node.js** for the backend and **Socket.IO/WebSockets** for real-time data transmission, allowing messages to be sent and received instantly. The front-end is designed to be intuitive and responsive, making it accessible across various devices. Additional functionalities such as emoji support, media sharing, private messaging, and user status indicators enhance user interaction and engagement.

This project not only demonstrates effective use of real-time communication protocols but also emphasizes secure, scalable, and maintainable code practices. It can be deployed in educational institutions, corporate environments, or social platforms where group communication is essential. Overall, the Group Live Chat Application serves as a practical and extensible solution for real-time group interaction needs.

1.INTRODUCTION

In today's fast-paced digital era, real-time communication plays a crucial role in both personal and professional environments. A Group Live Chat Application serves as a modern solution to facilitate instant, seamless, and efficient communication among multiple users within a shared platform.

Whether it's for educational collaboration, workplace discussions, social networking, or community building, group chat applications provide a centralized medium where users can exchange messages, media, and information instantly.

This application enables multiple users to interact in real-time within common chat rooms or channels. It supports features such as message broadcasting, typing indicators, time stamps, file sharing, emojis, and user authentication. Designed with user-friendliness and reliability in mind, the group chat system ensures synchronized communication and enhances user engagement through a responsive and interactive interface.

The primary goal of this project is to provide a scalable and secure environment where individuals can communicate freely and effectively, thus fostering better teamwork, faster decision-making, and improved connectivity.

1.1Motivation

The motivation behind developing a **Group Live Chat Application** stems from the increasing need for instant, reliable, and interactive communication tools in our digitally connected world. Traditional communication methods like emails and SMS are often too slow or limited for dynamic group interactions. In contrast, real-time chat systems provide a platform where users can collaborate, share ideas, and solve problems quickly and efficiently.

With the rise of remote work, online education, virtual communities, and digital businesses, the demand for effective group communication tools has grown significantly. A group chat application not only bridges the gap between users separated by distance but also fosters a sense of collaboration and community. It enables users to create chat rooms, discuss in real time, and share media, documents, and updates effortlessly.

Moreover, building such an application provides a practical opportunity to implement and understand key technologies such as socket programming, real-time data transmission, server-client architecture, user authentication, and UI/UX design. It serves as both a useful tool for users and a valuable learning experience for developers.

1.2Problem Definition

In modern digital communication, there is a growing demand for efficient, real-time interaction among multiple users, especially in educational institutions, organizations, and online communities. Existing communication platforms often come with limitations such as high cost, lack of customization, complexity, or inadequate support for group-based interactions. These challenges hinder effective collaboration, timely decision-making, and seamless information exchange.

The problem, therefore, is to **design and develop a user-friendly, secure, and scalable Group Live Chat Application** that allows multiple users to communicate in real-time within a shared environment. The system should support essential features like message broadcasting, real-time updates, user management, typing indicators, media sharing, and room-based chat segregation. Additionally, it must ensure data integrity, low latency, and ease of access across various devices.

This application aims to solve the communication gap in group settings by offering a centralized and interactive platform tailored for real-time discussions and collaboration.

1.3Existing System

Several group live chat applications already exist in the market, each serving different user needs and industries. Popular examples include:

1. **WhatsApp** – A widely used messaging app that supports group chats, media sharing, and voice/video calls. However, it's primarily mobile-focused and not ideal for large-scale or professional collaboration.

2. **Slack** – Designed for workplace communication, Slack offers channels (group chats), direct messages, file sharing, and integration with other tools. It's powerful but can be expensive and overwhelming for small teams or non-corporate users.

3. **Microsoft Teams** – Integrated with Microsoft 365, Teams is suited for corporate environments. It includes chat, video conferencing, and document collaboration. However, it requires a Microsoft ecosystem and may be too feature-heavy for simple group communication needs.

4. **Discord** – Originally for gamers, Discord now supports diverse communities with features like voice channels, role management, and group text chats. While feature-rich, its user interface may not be ideal for professional or academic settings.

5. **Telegram** – Offers group chats, bots, and media sharing with a focus on privacy and speed. However, customization and advanced chat room controls are limited.

1.4Proposed System

The proposed system is a real-time **Group Live Chat Application** designed to enable seamless communication among multiple users within shared chat rooms. Unlike existing platforms that are often complex or limited by cost and customization options, this application focuses on delivering a lightweight,

secure, and user-friendly chatting experience. It allows users to create or join group chat rooms, exchange messages in real time, and enjoy features such as typing indicators, message timestamps, emoji support, and media sharing. A secure login system ensures that only authorized users can access the platform, and all messages are stored in a database for future reference. The user interface is designed to be clean, intuitive, and responsive, making it accessible on both desktop and mobile devices. By providing essential chat features in a customizable and scalable package, the proposed system serves as an ideal solution for educational institutions, project teams, and online communities looking for an efficient way to stay connected and collaborate effectively.

1.5Requirements Specification

1.5.1 Software Requirements:

- **Operating System:** Windows or macOS
- **IDE (Integrated Development Environment):** Visual Studio Code
- **Programming Language:** JavaScript / HTML/CSS
- **Web Framework (if applicable):** Socket.IO / Node.js
- **Database:** SQLite / MySQL / MongoDB (based on application design)
- **Browser (for testing and usage):** Google Chrome, Microsoft Edge

1.5.2 Hardware Requirements:

- Processor: Multi-Core CPU
- **Processor Speed:** 1.76 GHz or above
- **Memory (RAM):** 8 GB to 16 GB
- **Storage:** 256 GB to 512 GB SSD
- **Network:** Stable internet connection for real-time communication

1.6Languages Used

1.6.1 HTML (Hyper Text Markup Language:

HTML is the backbone of any webpage, providing the essential structure and content. It uses a series of elements and tags to define headings, paragraphs, links, images, and other fundamental components of a webpage. HTML outlines the framework of the site, ensuring that content is organized and accessible to users and search engines alike.

1.6.2 CSS (Cascading Style Sheets):

CSS is used to style and layout web pages. While HTML provides the structure, CSS is responsible for the visual presentation. It controls the colors, fonts, spacing, and positioning of elements on the page, making the site visually appealing and user-friendly. CSS also allows for responsive design, ensuring that webpages look and function well on a variety of devices and screen sizes.

1.6.3 Node js :

Node.js is a powerful, open-source, server-side runtime environment built on Google Chrome's V8 JavaScript engine. It enables developers to build scalable and high-performance network applications, particularly real-time applications like chat systems. Node.js uses an event-driven, non-blocking I/O model, making it efficient and suitable for data-intensive applications that require continuous communication, such as live chat. Its package ecosystem, npm (Node Package Manager), provides access to thousands of reusable libraries and modules, including tools like Express.js for server-side logic and Socket.IO for real-time communication.

2.LITERATURE SURVEY

Shin, D., & Biocca, F. "Immersive Communication: Telepresence and Real-Time Interaction in Virtual Environments, 2020" .

This study, published in *Computers in Human Behavior*, explores how real-time interaction technologies like chat systems and video calls enhance the sense of presence and immersion in virtual environments. The research emphasizes the psychological and technical aspects of live communication, noting that immediate feedback and presence awareness (e.g., typing indicators and message receipts) improve user satisfaction and collaboration. However, the study also acknowledges challenges in maintaining low-latency performance and ensuring security, especially in large-scale systems.

Patel, S., & Singh, R. "Design and Implementation of Real-Time Chat Application Using Node.js and Socket.IO, 2019" .

Published in the *International Journal of Computer Applications*, this paper details the development of a real-time chat application using Node.js and Socket.IO. The methodology involves using non-blocking I/O and WebSocket technology to establish two-way communication between client and server. The system efficiently handles real-time data transmission and supports chat room creation, user presence, and message broadcasting. While it achieves low latency and high scalability, the authors note challenges in implementing persistent data storage and ensuring security for user sessions.

Mahajan, A., & Poonia, R. C. "Real-Time Web Chat Application using Node.js, Express, and MongoDB, 2020" .

This research, published in the *International Research Journal of Engineering and Technology (IRJET)*, presents the development of a group chat application with real-time features. The backend is built using Node.js and Express.js, while MongoDB is used for storing user credentials and chat logs. The application supports multiple chat rooms, user authentication, and real-time message updates via Socket.IO. The study highlights the advantages of using a NoSQL database for flexibility and faster retrieval, although it also points out that managing concurrent connections and ensuring secure data flow are critical challenges.

Alsaqour, R. A. et al. "Performance Evaluation of Web-Based Chat Application Under Varying Load Conditions, 2018" .

Published in the *Journal of Theoretical and Applied Information Technology*, this paper evaluates the performance of web chat applications under different user loads. The methodology includes stress testing chat servers built with Node.js and analyzing response times, packet loss, and CPU usage. The study concludes that Node.js is highly efficient for real-time communication, though performance degrades beyond a certain threshold without horizontal scaling. The research also stresses the importance of optimization in database interactions and WebSocket management for large-scale deployments.

Kumar, S., & Jaiswal, M. "Secure Group Chat Application using End-to-End Encryption, 2017" .

In this study published in the *International Journal of Security and Its Applications*, the authors focus on implementing end-to-end encryption in group chat applications to ensure privacy. The paper outlines encryption mechanisms, including RSA and AES, integrated into chat systems to protect messages from interception. The benefit of this approach is a high level of data confidentiality, but the downside includes increased processing overhead and complexity in managing keys for group participants, especially in dynamic chat rooms.

Yadav, P., & Gupta, A. "Enhancing User Interaction in Live Chat Applications through UI/UX Design, 2019" .

This paper, published in the *International Journal of Human–Computer Interaction*, explores the impact of UI/UX elements in live chat applications. The authors argue that features like smooth animations, user avatars, dark/light themes, and responsive layouts significantly enhance user engagement. Their methodology involves usability testing and heuristic evaluation of various chat UIs. While visually engaging interfaces boost user satisfaction, the study notes that balancing aesthetics with performance and device compatibility is a continuous challenge for developers.

Table 2.1: LITERATURE SURVEY

S.No	Year	Author(s)	Title	Techniques	Advantages	Disadvantages
1	2020	▪ Shin D. ▪ Biocca F.	*Immersive Communication: Telepresence and Real-Time Interaction in Virtual Environments*	▪ Real-time interaction with presence indicators ▪ Enhancing user immersion in communication systems	▪ Increases user engagement and realism ▪ Promotes collaboration in virtual spaces	▪ Requires high-speed internet and optimized backend ▪ Latency may impact user experience
2	2019	▪ Patel S. ▪ Singh R.	*Design and Implementation of Real-Time Chat Application Using Node.js and Socket.IO*	▪ Node.js and Socket.IO for real-time messaging ▪ Non-blocking event-driven architecture	▪ Supports multiple concurrent users efficiently ▪ Low latency communication	▪ Lacks built-in persistent message storage ▪ Needs strong security for live connections
3	2020	▪ Mahajan A. ▪ Poonia R.C.	*Real-Time Web Chat Application using Node.js, Express, and MongoDB*	▪ Backend using Node.js and Express ▪ MongoDB for data persistence ▪ WebSocket for messaging	▪ Real-time chatting with storage ▪ Scalable NoSQL structure	▪ Requires advanced backend logic ▪ Complex user management
4	2018	▪ Alsaqour R. A. ▪ et al.	*Performance Evaluation of Web-Based Chat Application Under Varying Load Conditions*	▪ Load testing on Node.js chat apps ▪ Monitoring CPU, latency, and packet loss	▪ Measures performance under stress ▪ Helps in scalability planning	▪ Performance drops at high load ▪ Requires optimization and scaling
5	2017	▪ Kumar S. ▪ Jaiswal M.	*Secure Group Chat Application using End-to-End Encryption*	▪ AES/RSA encryption for secure messaging ▪ Secure user sessions	▪ Ensures privacy of chat content ▪ Prevents data leakage	▪ Adds complexity to implementation ▪ Key management for groups is difficult
6	2019	▪ Yadav P. ▪ Gupta A.	*Enhancing User Interaction in Live Chat Applications through UI/UX Design*	▪ Usability testing ▪ Implementation of themes and animations	▪ Improves user satisfaction ▪ Makes chat more engaging and modern	▪ UI design can affect performance ▪ Compatibility issues on older devices

3.DESIGN METHODOLOGY

The design methodology for the Group Live Chat Application follows a **modular and iterative development approach** that ensures scalability, maintainability, and real-time responsiveness. The major stages are outlined below:

3.1 Technologies Used

The Group Live Chat Application is developed using a modern technology stack that supports real-time communication, modular development, and responsive user interface design. The key technologies used are:

Node.js: Building the Server-Side Logic and Real-Time Communication
Node.js is a powerful, event-driven JavaScript runtime built on Chrome's V8 engine, designed for developing scalable network applications.

- Non-Blocking I/O: Node.js operates on a non-blocking, asynchronous model, allowing it to handle multiple concurrent users efficiently—ideal for chat applications.
- Built-in HTTP Module: It enables quick setup of web servers and routing without the need for external libraries.
- Real-Time Capability: Integrated with Socket.IO, Node.js can push real-time messages and updates to clients, enabling instant chat message delivery.
- NPM (Node Package Manager): Provides access to a wide range of open-source packages and libraries which streamline development (e.g., Express, CORS, dotenv).
- Scalability: Due to its event-loop mechanism, Node.js can scale efficiently for thousands of concurrent connections, making it suitable for group chat applications.

HTML: Structuring the Chat Application Interface
HTML is used to provide the semantic and structural layout of the chat application's frontend.

- Semantic Tags: Tags like <header>, <section>, and <main> are used to organize chat layout components cleanly and meaningfully.
- Form Controls: Login forms, message input fields, and file upload elements are created using HTML5's improved form inputs.
- Multimedia Integration: HTML5 supports native <audio> and <video> elements, which can be useful for media sharing in chat applications.
- Canvas and Drag-Drop APIs: Though optional, HTML5 supports rich media content handling for future integration like drawings or annotations.

CSS: Styling the User Interface and Ensuring Responsiveness
CSS enables responsive and attractive user interfaces across devices.

- Layout Management: Flexbox and CSS Grid are used to structure chat boxes, sidebars, and navigation panels dynamically.
- Media Queries: Allow the chat interface to adapt based on screen size (desktop, tablet, mobile).
- Transitions and Animations: Smooth visual effects are added for message delivery, typing indicators, hover states, etc.
- Dark/Light Mode: CSS custom properties enable toggling themes based on user preference.

- Custom Fonts & Icons: Enhances the look of usernames, message bubbles, and UI buttons using external fonts and icon packs (e.g., FontAwesome).

MongoDB: Managing User Data and Chat Histories
MongoDB is a flexible NoSQL database used for backend data storage.

- Document-Oriented Storage: Stores chat messages, users, and room metadata as JSON-like documents.
- Schema Flexibility: Allows dynamic data models, helpful for evolving features like media attachments or message status (seen/delivered).
- Real-Time Data Sync: Works efficiently with Node.js and Socket.IO to reflect real-time chat data in the frontend.
- Scalability: Designed for distributed data processing, making it suitable for applications expecting growth.
- Indexing and Search: Ensures fast retrieval of users, message history, or media files.

3.2Flow Chart diagram:

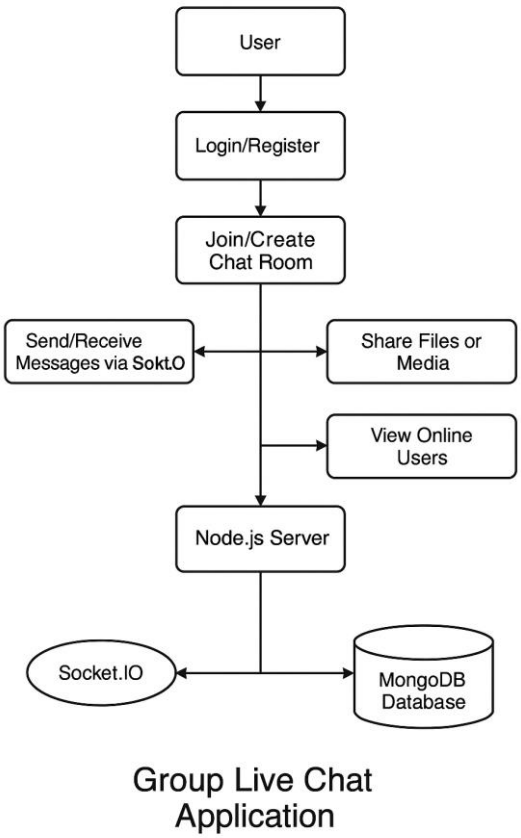


Figure 3.2: Flow Chart Diagram for Group Live Chat Application

This is the Design Phase Diagram for Group Live Chat Application. The above phases are carried out while creating the webpage.

3.3 Working Application Architecture:

3.3.1 Components

- Main Application Shell:

The base structure of the chat interface which includes login/register pages, chat room window, and sidebar for user lists or rooms.

- Navigation Component:

A persistent header or sidebar that allows users to navigate between available chat rooms, profile settings, or logout.

- Chat Window (Message Container):

The core interface where live chat happens. Messages are dynamically rendered in real-time using Socket.IO.

- User List Panel:

Displays the list of online users and participants in a chat room. Updates live as users join or leave.

- Message Input Area:

A text input area with emoji support, file upload button, and a send button. Sends data to the server over a WebSocket connection.

3.3.2 Data Flow

- Client-Side State Management:

JavaScript manages current user data (username, room), chat history, and typing states. LocalStorage can store login sessions for persistent access.

- WebSocket Communication:

Socket.IO enables real-time, bidirectional data flow. Clients send/receive messages, typing indicators, file alerts, and user status through WebSocket events.

- Data Persistence:

MongoDB is used to persist user details, chat messages, and file URLs for long-term access and backup.

3.3.3 Main Application Shell

- HTML Structure:

A base template that includes <header> for navigation, <main> for chat interaction, and a <footer> for copyright.

- CSS Styling:

Responsive layout using Flexbox and Grid for fluid resizing across devices (desktop, tablet, mobile). Theme switch (light/dark) is also handled here.

- JavaScript Initialization:

Initial script loads the UI, connects to the server via Socket.IO, and fetches user session data from LocalStorage if available.

3.3.4 Navigation Component

- HTML Structure:

A vertical/horizontal navigation bar with clickable icons or links like “Rooms,” “Profile,” “Logout.”

- CSS Styling:

Highlighted states for active links, hover effects, and smooth transitions using CSS3.

- JavaScript Functionality:

Event listeners capture navigation events (e.g., switching chat rooms) and load the corresponding content

dynamically without a full page reload.

3.3.5 Chat Window (Game Container Equivalent)

- HTML Structure:

A scrollable container dynamically rendering messages with timestamps, sender name, and optional media.

- CSS Styling:

Chat bubbles with different styles for sender/receiver, emojis, and attached media with preview thumbnails.

- JavaScript Functionality:

Dynamically appends new messages, manages scroll behavior, handles media previews, and integrates emoji selectors and typing notifications.

3.4.User Interface

3.4.1 Login/Registration Page

- Purpose: Allows users to securely create an account or log in to access chat functionalities.
- Components:
  - Input fields for username, email, and password.
  - Buttons for login, register, and password reset.
  - Basic validation and feedback messages.
- Design Features:
  - Responsive form layout.
  - Simple and modern styling using CSS3.
  - Error/success feedback with alerts.

3.4.2 Home Dashboard

- Purpose: Landing screen after login showing available chat rooms and online users.
- Components:
  - List of public and private chat rooms.
  - “Create Room” or “Join Room” options.
  - Online users section.
- Design Features:
  - Sidebar navigation.
  - Icons for rooms and user status (online/offline).

- Search bar for filtering users/rooms.

3.4.3 Chat Room Interface

- Purpose: Main area for real-time group communication.
- Components:
  - Chat Window: Scrollable area displaying sent and received messages.
  - User List Panel: Shows participants in the room.
  - Input Area: Textbox with emoji support, file upload button, and send button.
- Design Features:
  - Message bubbles with timestamps and sender names.
  - Auto-scroll and smooth animation effects.
  - Typing indicator with “User is typing...” message.
  - Support for image/file previews before sending.

3.4.4 Media and File Sharing

- Purpose: Enable users to share images, videos, and documents during chat.
- Components:
  - File input button with preview pane.
  - Upload progress indicator.
- Design Features:
  - Thumbnail previews for images.
  - Icons representing file types.
  - Download/view buttons for shared files.

4. IMPLEMENTATION AND RESULTS

Frontend Components

The frontend of Group Live Chat Application is built using HTML, CSS, and Node js, ensuring a responsive and interactive user interface.

Key components include:

5.1 Login/Register Page

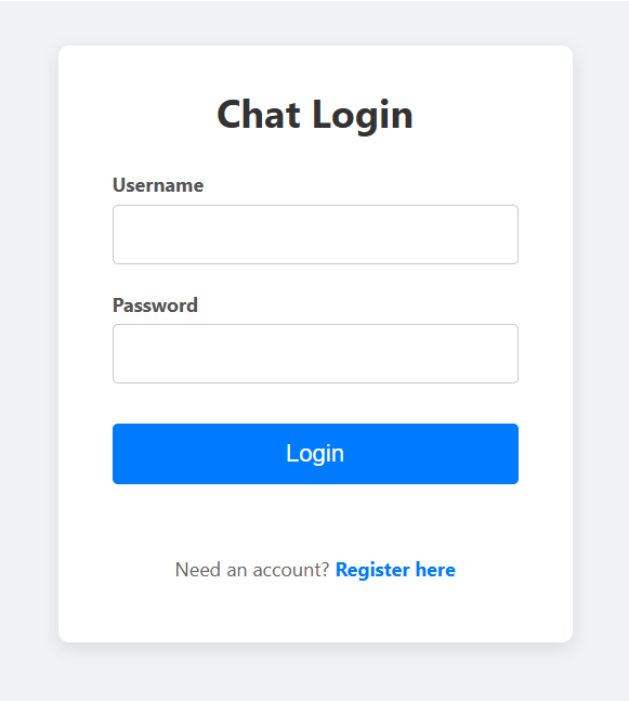


Figure 5.1: Login/Register Page of Group live Chat Application.

This is a simple and clean **chat login page** where users can enter their **username** and **password** to access the group chat application. It includes a login button and a registration link for new users. The design is minimal, user-friendly, and responsive.

5.2 Chat Room Page:

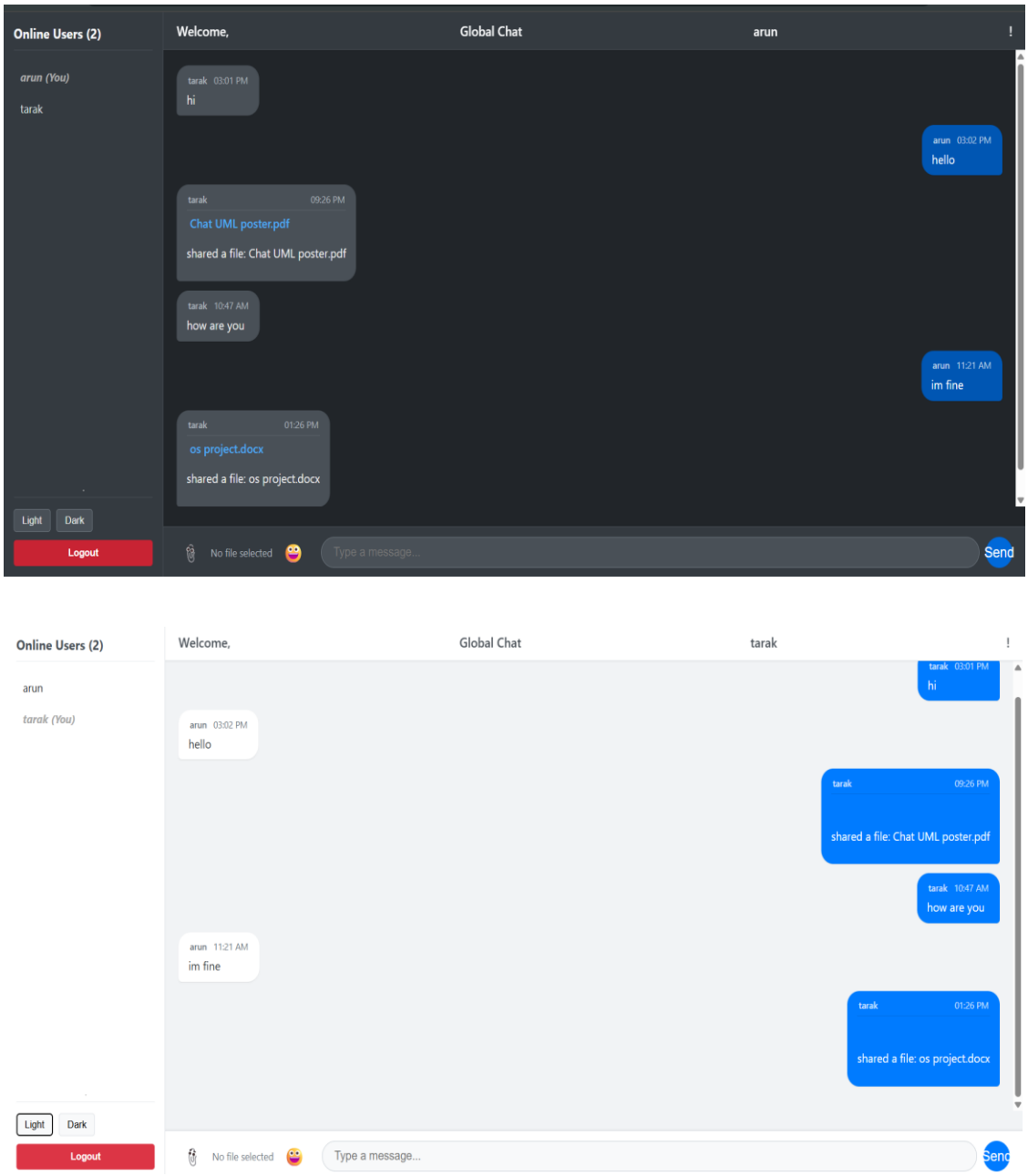


Figure 5.2: Chat Room page of Group Live Chat Application.

This is the main chat interface of the group live chat application. It displays online users, supports real-time messaging and file sharing, and includes options for theme switching (light/dark) and user logout. The layout is clean and user-friendly, enabling smooth group communication.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Group Live Chat Application is a robust and user-friendly platform designed to facilitate real-time communication among multiple users. Utilizing modern web technologies like Node.js, HTML5, CSS3, and MongoDB, the system ensures seamless data exchange, dynamic content rendering, and reliable data storage. The application integrates essential features such as user authentication, live messaging, file sharing, and theme switching, enhancing user experience and engagement.

Through a modular architecture and clearly defined components—including the main application shell, navigation system, and dynamic game/chat container—the project emphasizes scalability, maintainability, and responsiveness across devices. The chat interface promotes efficient interaction, making it suitable for collaborative use in educational institutions, professional teams, and social communities.

The project demonstrates the effective use of web technologies to build real-time applications and lays a strong foundation for further enhancement and scalability.

5.2 Future Scope

Looking forward, the application presents several opportunities for enhancement and wider adoption. Possible future developments include:

- Private and Group Chats: Enabling users to create private rooms or group discussions for more focused interactions.
- Voice and Video Chat Integration: Introducing multimedia communication capabilities for
- a more immersive experience.
- End-to-End Encryption: Ensuring secure communication by integrating advanced encryption protocols.
- Chatbot Assistance: Providing intelligent responses and support for user queries.
- Push Notifications: Keeping users informed about new messages or events in real time, even when offline.
- Mobile App Deployment: Expanding accessibility by developing native or hybrid mobile applications.

BIBLIOGRAPHY

1. Smith, John. *Interactive Mini-Games: Engaging Users on Web Platforms*. Journal of Interactive Entertainment, 2020.
2. Zhang, Taocan, Wang, Tiancheng, and Li, Shuqin. *Design and Implementation of Web Front End of Computer Game Platform Based on Electron*. Journal of Physics: Conference Series, 2019.
3. Murti, Heru Astikasari Setya, Hastjarjo, Thomas Dicky, and Ferdiana, Ridi. *Platform and Genre Identification for Designing Serious Games*. International Journal of Advanced Computer Science and Applications, 2019.
4. Song, Xiaojian. *Experience Generation in Tic-Tac-Toe for General Game Learning*. International Journal of Game-Based Learning, 2018.
5. Fisher, D.K., Leon, Nury, Fitzpatrick, A.J., and Wessen, Alice. *Playing Around in the Solar System: Mini-Games for Many Missions*. Journal of Spacecraft and Rockets, 2018.
6. Guo, Kaihua. *Design and Management of Web Game Interactive Application Platform Based on RETAIN Model*. International Journal of Online Engineering (iJOE), 2017.
7. Flanagan, David. *JavaScript: The Definitive Guide*. O'Reilly Media, 2020.
8. Duckett, Jon. *HTML and CSS: Design and Build Websites*. Wiley, 2011.
9. MongoDB Inc. *MongoDB Documentation*. Available at: https://www.mongodb.com/docs/
10. Node.js Foundation. *Node.js Documentation*. Available at: https://nodejs.org/en/docs/

Appendix

db.js :

```
// config/db.js
const mongoose = require('mongoose');
const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI, {
            serverSelectionTimeoutMS: 30000, // 30 seconds
        });
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error(`\n--- MongoDB Connection Error ---`);
        console.error(`ERROR: ${error.message}`);
        console.error(`\nPossible causes:`);
        console.error(`1. Incorrect MONGO_URI (check username, password, database name).`);
        console.error(`2. MongoDB Atlas IP Whitelist: Ensure your current public IP address is added.`);
        console.error(`3. Network issues or firewall blocking access to MongoDB Atlas.`);
        console.error(`-----------------------------------\n`);
        process.exit(1); // Exit process with failure
    }
};
module.exports = connectDB;
```

Message.js :

```
// models/Message.js
const mongoose = require('mongoose');
const MessageSchema = new mongoose.Schema({
    username: {
```

```
    type: String,
    required: true,
},
userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
},
text: {
    type: String,
    required: false,
},
timestamp: {
    type: Date,
    default: Date.now,
},
isPrivate: { // Flag to identify PMs
    type: Boolean,
    default: false
},
senderUsername: {
    type: String,
    required: false // Not required for global messages
},
senderId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: false // Not required for global messages
},
```

```
    recipientUsername: { // Recipient's username
        type: String,
        index: true, // Index for potentially faster lookup
        required: false // Not required for global messages
    },
    recipientId: { // Recipient's User ID
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        index: true,
        required: false // Not required for global messages
    },
    fileUrl: {
        type: String,
        required: false // Only present if the message is a file
    },
    fileName: {
        type: String,
        required: false // Only present if the message is a file
    },
    fileMimeType: {
        type: String,
        required: false // Only present if the message is a file
    }
});
MessageSchema.index({ isPrivate: 1, userId: 1, recipientId: 1, timestamp: -1 });
MessageSchema.index({ isPrivate: 1, recipientId: 1, userId: 1, timestamp: -1 });
module.exports = mongoose.model('Message', MessageSchema);
```

User.js :

```
// models/User.js
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const SALT_WORK_FACTOR = 10; // Cost factor for hashing
const UserSchema = new mongoose.Schema({
    username: {
        type: String,
        required: true,
        unique: true,
        trim: true, // Remove whitespace
        lowercase: true // Store usernames consistently
    },
    password: { // Store hashed passwords only!
        type: String,
        required: true,
    },
    isAdmin: {
        type: Boolean,
        default: false,
    },
    createdAt: {
        type: Date,
        default: Date.now,
    },
});
UserSchema.pre('save', function(next) {
    const user = this;
    if (!user.isModified('password')) return next();
    bcrypt.genSalt(SALT_WORK_FACTOR, function(err, salt) {
        if (err) return next(err);
```

```
        bcrypt.hash(user.password, salt, function(err, hash) {
            if (err) return next(err);
            user.password = hash;
            next();
        });
    });
});
UserSchema.methods.comparePassword = function(candidatePassword) {
    return bcrypt.compare(candidatePassword, this.password);
};
module.exports = mongoose.model('User', UserSchema);
```

index.html :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Real-Time Chat</title>

    <link rel="stylesheet" href="style.css">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/picmo@latest/dist/styles.css"/>
    <script src="/socket.io/socket.io.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/picmo@latest/dist/umd/picmo.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/@picmo/renderer-twemoji@latest/dist/umd/picmo-renderer-twemoji.js"></script>
</head>
<body>
    <div id="app-container">
        <div id="sidebar">
            <h3>Online Users (<span id="user-count">0</span>)</h3>
```

```
        <ul id="user-list"></ul>
        <hr>
        <div id="themes">
          <button data-theme="light">Light</button>
          <button data-theme="dark">Dark</button>
        </div>
      </div>
      <div id="chat-window">
        <div id="chat-header">
          Welcome, <span id="username-display">User</span>!
        </div>
        <ul id="messages"></ul>
        <div id="typing-indicator"></div>
        <form id="form" action="">
          <div class="file-input-container">
            <input type="file" id="fileInput" name="chatFile" style="display: none;" />
            <button type="button" id="selectFileButton" title="Attach File">📎</button>
            <span id="selectedFileName" class="file-name-display">No file selected</span>
          </div>
          <button type="button" id="emoji-button" title="Pick emoji">😀</button>
          <input id="input" autocomplete="off" placeholder="Type a message..." />
          <button type="submit">Send</button>
        </form>
      </div>
    </div>
    <script src="client.js"></script>
</body>
</html>
```

login.html :

```
<!DOCTYPE html>
<html>
<head>
    <title>Login - Real-Time Chat</title>
    <link rel="stylesheet" href="style.css"> <link rel="stylesheet" href="login.css"> </head>
<body>
    <div id="login-container">
        <h2>Chat Login</h2>
        <form id="login-form">
            <div class="form-group">
                <label for="username">Username</label>
                <input type="text" id="username" name="username" required>
            </div>
            <div class="form-group">
                <label for="password">Password</label>
                <input type="password" id="password" name="password" required>
            </div>
            <button type="submit">Login</button>
            <p id="error-message" class="error"></p>
             <p class="info">Need an account? <a href="#" id="show-register">Register here</a></p>
        </form>
        <form id="register-form" style="display: none;"> <h2>Register</h2>
             <div class="form-group">
                <label for="reg-username">Username</label>
                <input type="text" id="reg-username" name="username" required>
            </div>
            <div class="form-group">
                <label for="reg-password">Password</label>
```

```
            <input type="password" id="reg-password" name="password" required>
        </div>
        <button type="submit">Register</button>
        <p id="register-error-message" class="error"></p>
        <p id="register-success-message" class="success"></p>
        <p class="info">Already have an account? <a href="#" id="show-login">Login here</a></p>
    </form>
  </div>
  <script src="login.js"></script>
</body>
</html>
```

login.css :

```
/* public/login.css */
body {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f0f2f5; /* Light background specifically for login page body */
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; /* Consistent font */
   /* Override general body styles from style.css if needed */
   overflow: auto; /* Allow scrolling if content exceeds viewport */
}
#login-container {
  background-color: #fff;
  padding: 30px 40px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  text-align: center;
```

```
    width: 100%; /* Take full width on small screens */
    max-width: 380px; /* Max width on larger screens */
    box-sizing: border-box;
}
#login-container h2 {
    margin-top: 0; /* Remove default margin */
    margin-bottom: 25px;
    color: #333;
    font-size: 1.8em; /* Larger heading */
}
.form-group {
    margin-bottom: 20px;
    text-align: left;
}
.form-group label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
    color: #555;
    font-size: 0.9em;
}
.form-group input {
    width: 100%;
    padding: 12px; /* More padding */
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box; /* Include padding in width */
    font-size: 1em;
}
```

```
.form-group input:focus {
    border-color: #80bdff;
    outline: none;
    box-shadow: 0 0 0 0.2rem rgba(0,123,255,.25); /* Subtle focus glow */
}
button[type="submit"] {
   width: 100%;
   padding: 12px;
   background-color: #007bff;
   color: white;
   border: none;
   border-radius: 4px;
   cursor: pointer;
   font-size: 1.1em; /* Slightly larger button text */
   transition: background-color 0.3s ease;
   margin-top: 10px; /* Add some space above button */
}
button[type="submit"]:hover {
   background-color: #0056b3;
}
/* --- Error/Success/Info Messages --- */
.error, .success, .info {
    margin-top: 15px;
    min-height: 1.2em; /* Reserve space */
    font-size: 0.9em;
    line-height: 1.4;
}
.error {
   color: #d9534f; /* Red for errors */
```

```
}
.success {
    color: #5cb85c; /* Green for success */
}
.info {
    margin-top: 20px;
    color: #666;
}
 .info a {
    color: #007bff;
    text-decoration: none;
    font-weight: bold;
 }
 .info a:hover {
    text-decoration: underline;
 }
```

login.js :

```
// public/login.js
console.log("DEBUG: login.js script has started execution!"); // Keep this for debugging for now
// --- Get DOM Elements ---
const loginForm = document.getElementById('login-form');
const registerForm = document.getElementById('register-form');
const loginErrorMessage = document.getElementById('error-message');
const registerErrorMessage = document.getElementById('register-error-message');
const registerSuccessMessage = document.getElementById('register-success-message');
const showRegisterLink = document.getElementById('show-register');
const showLoginLink = document.getElementById('show-login');
// --- Toggle Forms ---
```

```
if (showRegisterLink && loginForm && registerForm) {
    showRegisterLink.addEventListener('click', (e) => {
        e.preventDefault(); // Prevent default link behavior
        loginForm.style.display = 'none';
        registerForm.style.display = 'block';
        if (loginErrorMessage) loginErrorMessage.textContent = ''; // Clear messages safely
        console.log("DEBUG: Switched to register form"); // Add debug log
    });
} else {
    console.error("DEBUG: Could not find showRegisterLink or form elements!");
}
if (showLoginLink && loginForm && registerForm) {
    showLoginLink.addEventListener('click', (e) => {
        e.preventDefault(); // Prevent default link behavior
        registerForm.style.display = 'none';
        loginForm.style.display = 'block';
        if (registerErrorMessage) registerErrorMessage.textContent = ''; // Clear messages safely
        if (registerSuccessMessage) registerSuccessMessage.textContent = '';
        console.log("DEBUG: Switched to login form"); // Add debug log
    });
} else {
     console.error("DEBUG: Could not find showLoginLink or form elements!");
}
// --- Login Handler ---
if (loginForm) {
    loginForm.addEventListener('submit', async (e) => {
        e.preventDefault(); // *** Crucial: Stops the default GET submission ***
        console.log("DEBUG: Login form submitted"); // Add debug log
        if (loginErrorMessage) loginErrorMessage.textContent = ''; // Clear previous errors
```

```
const usernameInput = loginForm.username;
const passwordInput = loginForm.password;
if (!usernameInput || !passwordInput) {
    console.error("DEBUG: Username or password input not found in login form");
    if(loginErrorMessage) loginErrorMessage.textContent = 'Form error. Please refresh.';
    return;
}
const username = usernameInput.value.trim().toLowerCase();
const password = passwordInput.value;
if (!username || !password) {
    if(loginErrorMessage) loginErrorMessage.textContent = 'Please enter username and password.';
    return;
}
try {
   console.log("DEBUG: Sending login request for user:", username); // Add debug log
   const response = await fetch('/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password }),
   });
   const data = await response.json(); // Try to parse JSON regardless of response.ok
   console.log("DEBUG: Login response status:", response.status, "Data:", data); // Add debug log
   if (!response.ok) {
      throw new Error(data.message || `Login failed with status ${response.status}`);
   }
   console.log("DEBUG: Login successful. Token:", data.token); // Add debug log
   localStorage.setItem('chat_token', data.token);
   localStorage.setItem('chat_username', data.username); // Store username too
   window.location.href = '/'; // Redirect to chat page (index.html)
```

```
        } catch (error) {
            console.error('Login error:', error); // Log the full error
            if(loginErrorMessage) loginErrorMessage.textContent = error.message || 'An error occurred during login.';
        }
    });
} else {
     console.error("DEBUG: Login form element not found!");
}
// --- Registration Handler ---
if (registerForm) {
    registerForm.addEventListener('submit', async (e) => {
        e.preventDefault(); // *** Crucial: Stops the default GET submission ***
        console.log("DEBUG: Register form submitted"); // Add debug log
        if (registerErrorMessage) registerErrorMessage.textContent = '';
        if (registerSuccessMessage) registerSuccessMessage.textContent = '';
        const usernameInput = registerForm['reg-username'];
        const passwordInput = registerForm['reg-password'];
        if (!usernameInput || !passwordInput) {
             console.error("DEBUG: Username or password input not found in register form");
              if (registerErrorMessage) registerErrorMessage.textContent = 'Form error. Please refresh.';
             return;
        }
        const username = usernameInput.value.trim().toLowerCase();
        const password = passwordInput.value;
        if (!username || !password) {
            if (registerErrorMessage) registerErrorMessage.textContent = 'Username and password are required.';
            return;
        }
        if (password.length < 6) {
```

```
            if (registerErrorMessage) registerErrorMessage.textContent = 'Password must be at least 6 characters long.';
            return;
        }
        try {
            console.log("DEBUG: Sending register request for user:", username); // Add debug log
            const response = await fetch('/register', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ username, password }),
            });
            const data = await response.json(); // Try to parse JSON regardless of response.ok
            console.log("DEBUG: Register response status:", response.status, "Data:", data); // Add debug log
            if (!response.ok) {
                throw new Error(data.message || `Registration failed with status ${response.status}`);
            }
            console.log("DEBUG: Registration successful."); // Add debug log
            if (registerSuccessMessage) registerSuccessMessage.textContent = 'Registration successful! You can now log in.';
            registerForm.reset(); // Clear the form
            console.error('Registration error:', error); // Log the full error
            if (registerErrorMessage) registerErrorMessage.textContent = error.message || 'An error occurred during registration.';
        }
    });
} else {
    console.error("DEBUG: Register form element not found!");
}
console.log("DEBUG: login.js script finished execution."); // Add debug log at the end
```

server.js :

```
// server.js (Complete File with Private Messaging and File Sharing)
```

```
const express = require('express');
const http = require('http');
const { Server } = require(''socket.io'');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
require('dotenv').config(); // Loads .env file variables
const path = require('path');
const connectDB = require('./config/db'); // Your database connection function
const Message = require('./models/Message'); // Mongoose Message model
const User = require('./models/User');      // Mongoose User model
const multer = require('multer'); // NEW: For handling file uploads
// --- Basic Setup ---
const app = express(); // Create the Express application
const server = http.createServer(app); // Create HTTP server for Socket.IO
const io = new Server(server); // Attach Socket.IO to the server
const PORT = process.env.PORT || 3000; // Define the port
// --- Database Connection ---
connectDB(); // Call the function to connect to MongoDB
// --- JWT Secret ---
const JWT_SECRET = process.env.JWT_SECRET || 'your-very-strong-secret-key-replace-me';
// --- Express Middleware ---
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
// --- Multer Configuration for File Uploads ---

const storage = multer.diskStorage({
    destination: function (req, file, cb) {
        cb(null, path.join(__dirname, 'uploads'));
```

```
    },
    filename: function (req, file, cb) {
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
        cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));
    }
});
const fileFilter = (req, file, cb) => {
    const allowedMimeTypes = [
        'image/jpeg',
        'image/png',
        'image/gif',
        'application/pdf',
        'application/msword', // .doc
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document', // .docx
        'application/vnd.ms-excel', // .xls
        'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet', // .xlsx
        'application/vnd.ms-powerpoint', // .ppt
        'application/vnd.openxmlformats-officedocument.presentationml.presentation', // .pptx
        'text/plain'
    ];
    if (allowedMimeTypes.includes(file.mimetype)) {
        cb(null, true); // Accept file
    } else {
        cb(new Error('Invalid file type. Only images, PDFs, and common document types are allowed.'), false); // Reject file
    }
};
const upload = multer({
    storage: storage,
    fileFilter: fileFilter,
```

```
    limits: {
        fileSize: 5 * 1024 * 1024 // Limit file size to 5MB (adjust as needed)
    }
});
// --- JWT Authentication Middleware for HTTP Routes ---
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1]; // Expects "Bearer TOKEN"
    if (!token) {
        console.warn('Authentication token missing for HTTP request.');
        return res.status(401).json({ message: 'Authentication token required.' });
    }
    jwt.verify(token, JWT_SECRET, (err, user) => {
        if (err) {
            console.error('JWT verification failed for HTTP request:', err.message);
            return res.status(403).json({ message: 'Invalid or expired token.' });
        }
        req.user = user; // Attach user payload (userId, username) to the request
        next();
    });
};
// --- HTTP Routes ---
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'public', 'index.html'));
});
// Route for the login page
app.get('/login', (req, res) => {
    res.sendFile(path.join(__dirname, 'public', 'login.html'));
});
```

```
// --- User Registration Endpoint ---

app.post('/register', async (req, res) => {

    const { username, password } = req.body;

    if (!username || !password) { /* ... validation ... */ return res.status(400).json({ message: 'Username and password required' }); }

    if (password.length < 6) { /* ... validation ... */ return res.status(400).json({ message: 'Password must be at least 6 characters' }); }

    try {

        const existingUser = await User.findOne({ username: username.toLowerCase() });

        if (existingUser) { return res.status(400).json({ message: 'Username already taken' }); }

        const newUser = new User({ username: username.toLowerCase(), password });

        await newUser.save();

        res.status(201).json({ message: 'User registered successfully' });

    } catch (error) { /* ... error handling ... */ console.error('Registration Error:', error); res.status(500).json({ message: 'Server error during registration' }); }

});

// --- User Login Endpoint ---

app.post('/login', async (req, res) => {

    const { username, password } = req.body;

    if (!username || !password) { /* ... validation ... */ return res.status(400).json({ message: 'Username and password required' }); }

    try {

        const user = await User.findOne({ username: username.toLowerCase() });

        if (!user) { /* ... validation ... */ return res.status(401).json({ message: 'Invalid credentials' }); }

        const isMatch = await user.comparePassword(password);

        if (!isMatch) { /* ... validation ... */ return res.status(401).json({ message: 'Invalid credentials' }); }

        const payload = { userId: user._id, username: user.username };

        const token = jwt.sign(payload, JWT_SECRET, { expiresIn: '1h' });

        res.json({ token, username: user.username });

    } catch (error) { /* ... error handling ... */ console.error('Login Error:', error); res.status(500).json({ message: 'Server error during login' }); }
```

```
});
// NEW: File Upload Endpoint
app.post('/upload', authenticateToken, upload.single('chatFile'), async (req, res) => {
    try {
        if (!req.file) {
            return res.status(400).json({ message: 'No file uploaded.' });
        }
        const senderUsername = req.user.username;
        const senderUserId = req.user.userId;
        const fileUrl = `/uploads/${req.file.filename}`; // URL to access the uploaded file
        const fileName = req.file.originalname;
        const fileMimeType = req.file.mimetype;
        console.log(`File uploaded by ${senderUsername}: ${fileName} (${fileMimeType})`);
        const message = new Message({
            username: senderUsername,
            userId: senderUserId,
            text: `shared a file: ${fileName}`, // A default text message indicating a file share
            timestamp: new Date(),
            isPrivate: false, // For now, assume global file sharing. Can be extended for PMs.
            fileUrl: fileUrl,
            fileName: fileName,
            fileMimeType: fileMimeType
        });
        const savedMessage = await message.save();
        io.emit('chat_message', savedMessage);
        res.status(200).json({
            message: 'File uploaded successfully!',
            file: {
                url: fileUrl,
```

```
            name: fileName,
            mimeType: fileMimeType
        }
    });
  } catch (error) {
    if (error instanceof multer.MulterError) {
        // Multer-specific errors (e.g., file size limit exceeded)
        console.error('Multer error during file upload:', error.message);
        return res.status(400).json({ message: `File upload failed: ${error.message}` });
    } else if (error.message.includes('Invalid file type')) {
        // Custom file filter error
        console.error('File type error:', error.message);
        return res.status(400).json({ message: error.message });
    } else {
        console.error('Server error during file upload:', error);
        res.status(500).json({ message: 'Server error during file upload.' });
    }
  }
});
// --- Socket.IO Authentication Middleware ---
io.use((socket, next) => {
  const token = socket.handshake.auth.token;
  if (!token) { /* ... error handling ... */ console.error('Auth Error: No token provided by', socket.id); return next(new Error('Authentication error: No token')); }
  jwt.verify(token, JWT_SECRET, (err, decoded) => {
    if (err) { /* ... error handling ... */ console.error('Auth Error: Invalid token for', socket.id, err.message); return next(new Error('Authentication error: Invalid token')); }
    socket.user = decoded; // Attach user data to socket
    console.log(`Auth Success: User ${socket.user.username} (${socket.user.userId}) authenticated for socket ${socket.id}`);
```

```
        next();
    });
});
let usersMap = new Map();
// --- Socket.IO Connection Handling ---
io.on('connection', (socket) => {
    const username = socket.user.username;
    const userId = socket.user.userId;
    console.log(`User connected: ${username} (${userId}) with socket ${socket.id}`);
    // --- Handle User Presence ---
    usersMap.set(username, { socketId: socket.id, userId: userId });
    const onlineUsernames = Array.from(usersMap.keys());
    io.emit('update_user_list', onlineUsernames); // Send updated list to ALL clients
    socket.emit('your_details', { username: username, userId: userId }); // Send user their details
    // --- Load Recent Global Chat History ---
    Message.find({ isPrivate: false }).sort({ timestamp: -1 }).limit(50)
        .then(messages => { socket.emit('load_history', messages.reverse()); })
        .catch(err => console.error('Error loading global chat history:', err));
    // --- Handle Incoming Global Chat Messages ---
    socket.on('chat_message', async (msgText) => {
        if (!msgText || typeof msgText !== 'string' || msgText.trim().length === 0) { /* ... validation ... */ return; }
        const text = msgText.trim();
        console.log(`Global Message from ${username}: ${text}`);
        const message = new Message({
            username: username, // Sender username
            userId: userId,     // Sender user ID
            text: text,
            timestamp: new Date(),
            isPrivate: false // Explicitly false for global messages
```

```
    });
    try {
        const savedMessage = await message.save();
        io.emit('chat_message', savedMessage); // Broadcast global message to everyone
    } catch (err) {
        console.error('Error saving global message:', err);
    }
});
// --- Handle Private Messages ---
socket.on('private_message', async ({ recipientUsername, text }) => {
    const senderUsername = socket.user.username;
    const senderUserId = socket.user.userId;
    if (!recipientUsername || !text || typeof text !== 'string' || text.trim().length === 0) {
        console.warn(`Invalid PM body from ${senderUsername} to ${recipientUsername}`);
        socket.emit('pm_error', { message: 'Cannot send empty message.' }); return;
    }
    if (recipientUsername === senderUsername) {
        console.warn(`User ${senderUsername} tried to PM self.`);
        socket.emit('pm_error', { message: "You can't send a private message to yourself." }); return;
    }
    text = text.trim();
    const recipientData = usersMap.get(recipientUsername); // Find recipient
    if (recipientData) { // Recipient online
        const recipientSocketId = recipientData.socketId;
        const recipientUserId = recipientData.userId;
        console.log(`Relaying PM from ${senderUsername} to ${recipientUsername} (${recipientSocketId})`);
        const messageData = { // Prepare message data
            username: senderUsername,
            userId: senderUserId,
```

```
            text: text,
            senderUsername: senderUsername,
            senderId: senderUserId,
            recipientUsername: recipientUsername,
            recipientId: recipientUserId,
            timestamp: new Date(),
            isPrivate: true
        };
        const message = new Message(messageData); // Create message model instance
        try {
            const savedMessage = await message.save(); // Save to DB
            io.to(recipientSocketId).emit('private_message', savedMessage);
            socket.emit('private_message', savedMessage); // Send back to sender
        } catch (err) {
            console.error('Error saving private message:', err);
            socket.emit('pm_error', { message: 'Server failed to save message.' });
        }
    } else { // Recipient offline
        console.log(`User ${recipientUsername} not online for PM from ${senderUsername}.`);
        socket.emit('pm_error', { recipientUsername: recipientUsername, message: `${recipientUsername} is currently offline.` });
    }
});
// --- Handle Typing Indicators --- (Could be enhanced for PMs later)
socket.on('typing', () => { socket.broadcast.emit('user_typing', username); });
socket.on('stopped_typing', () => { socket.broadcast.emit('user_stopped_typing', username); });
// --- Handle Disconnect ---
socket.on('disconnect', (reason) => {
    console.log(`User disconnected: ${username} (${socket.id}). Reason: ${reason}`);
    if (usersMap.has(username) && usersMap.get(username).socketId === socket.id) {
```

```
            usersMap.delete(username);
            const updatedOnlineUsernames = Array.from(usersMap.keys());
            io.emit('update_user_list', updatedOnlineUsernames);
            socket.broadcast.emit('user_stopped_typing', username);
            console.log(`${username} removed from online users map.`);
        } else { console.log(`${username} disconnect event for socket ${socket.id}, but user map has different/no entry.`); }
    });
}); // End io.on('connection')
// --- Start Server ---
server.listen(PORT, () => {
    console.log(`Server listening on http://localhost:${PORT}`);
});
```